Lab 5

The Server Room Heist: Joins and Aggregates

CSE 4308

DATABASE MANAGEMENT SYSTEMS LAB

Prepared by:
Samnun Azfar, Jr. Lecturer, CSE
Samia Nawsheen, Jr. Lecturer, CSE
Academic Year: AY25-26

Contents

1 Investigative Database Schema **5**

2 Technique 1: Connecting Evidence (SQL Joins) **7**
2.1 INNER JOIN (Confirmed Presence) 7
2.2 NATURAL JOIN (The Implicit Connection) 7
2.3 Joining Three Tables 8

3 Technique 2: Finding Missing Links **9**
3.1 LEFT JOIN 9
3.2 RIGHT JOIN 9

4 Technique 3: Profiling (Aggregates) **9**
4.1 COUNT, SUM, AVG, MAX 9

5 Technique 4: Grouping Suspects (GROUP BY) **10**
5.1 GROUP BY with Aggregate 10
5.2 GROUP BY with JOIN (Readable Report) 10

6 Technique 5: Narrowing the List (HAVING) **10**
6.1 HAVING Clause 10

7 Lab Tasks **11**

Lab Workflow

Repeat this subsection for every single lab.

1. Connect to the internet by running nmtui in terminal, add a dsl connection, then go back and activate that dsl connection. Make sure that your net id is not being used in any other pc or router, if you face any issue, use any other net id from your friends which is not being used.

2. Download all the files posted in the google classroom.

3. Disconnect from the **dsl** connection via nmtui, connect to the ethernet/wired connection via nmtui. If no such connection exists, create an **ethernet** connection and then activate.

4. **Create Your Role and Database**

   Inside the PostgreSQL shell, create your own role (user) and database.

   1. In the Linux terminal:

```
psql
```

   Expected prompt:

```
cse=#
```

   2. Create Your Role and Database

   Copy and run the following commands inside PostgreSQL:

```
-- 1. Terminate existing database connections
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'cse4308_12345'
  AND pid <> pg_backend_pid();

-- 2. Remove existing database
DROP DATABASE IF EXISTS cse4308_12345;

-- 3. Remove existing role
DROP ROLE IF EXISTS student_12345;

-- 4. Create role
CREATE ROLE student_12345 LOGIN;

-- 5. Set password
ALTER ROLE student_12345
WITH PASSWORD 'student12345';

-- 6. Create database
CREATE DATABASE cse4308_12345
OWNER student_12345;

-- 7. Grant privileges
GRANT ALL PRIVILEGES
ON DATABASE cse4308_12345
TO student_12345;
```

3. Exit PostgreSQL:

```
\q
```

5. **Connect Using Your Newly Created User**

1. Connect to Your Database:

Use the following command to log in as the `student_12345` user:

```
psql -h localhost -U student_12345 -d cse4308_12345
```

2. Verify the Connection:

Verify that you are connected to the correct database with the following command:

```
\conninfo
```

Expected output:

```
Database: cse4308_12345, User: student_12345
```

3. Exit PostgreSQL:

```
\q
```

6. **Load the Lab SQL File (MUST be run outside psql)**

Load the SQL file that sets up the tables and inserts initial data.

1. Load the SQL File:

Run the following command to load the lab SQL file:

```
psql -h localhost -U student_12345 -d cse4308_12345 -f /home/cse/Downloads/
 lab_05_setup.sql
```

This will: - Drop old tables (if they exist). - Create new tables. - Insert initial rows of data.

7. **Reconnect and Verify Tables**

After loading the SQL file, reconnect to PostgreSQL to verify that the tables have been created.

1. Reconnect to Your Database:

```
psql -h localhost -U student_12345 -d cse4308_12345
```

2. Check Tables:

Verify the tables have been created by listing them:

```
\dt
```

3. View Table Data:

Verify the data by running the following SQL queries:

```
SELECT * FROM <table_name>;
```

8. Open a text editor and save it in the appropriate submission name format `fullid_CSE4308_L5_T1.sql`. You will use this text editor to write your solutions, make sure to save it after writing each solution. The structure of this file will be:

```
-- Task 1
INSERT INTO students (student_id, name, major, gpa)
VALUES (104, 'David', 'ME', 3.60);
--Task 2
SELECT name, gpa FROM students WHERE major = 'CSE';
```

1 Investigative Database Schema

At University X, the Server Room houses sensitive exam questions and research data. On December 7, 2026, a security breach occurred. Someone physically entered the Server Room and deployed a script to leak data. The physical security logs, cafeteria receipts, and library records are the only evidence left behind. You are the Lead Database Detective hired to identify the culprit among the staff and students. You must do the following to solve the tasks:

1. Intelligence Gathering
2. Connecting the Dots based on gathered info. Cross-reference the Suspects with the Access Logs and Locations.
3. Look for anomalies. Who has never borrowed a book? Who has never ordered food? Who has no access logs at all?
4. Behavioral Analysis among the suspects
5. Follow this trail of breadcrumbs and catch the hacker before they wipe the database logs!

To solve the crime, you must understand the relationships between the entities on campus. We will use the following three sample tables for the lecture examples: **suspects**, **locations**, and **access_logs**.

Table 1: suspects

suspect_id (PK)	**name**	**occupation**	**access_level**
1	Zahid	Exam Controller	5
2	Nafis	Student	1
3	Tabib	Research Assistant	4
4	Sieam	Student	1

Table 2: locations

location_id (PK)	**location_name**	**floor_number**
101	Server Room	3
102	North Canteen	1
103	Library	2

Table 3: access_logs

log_id (PK)	suspect_id (FK)	location_id (FK)	entry_time
901	1	101	14:00
902	2	102	14:10
903	1	102	15:00
904	3	101	09:00

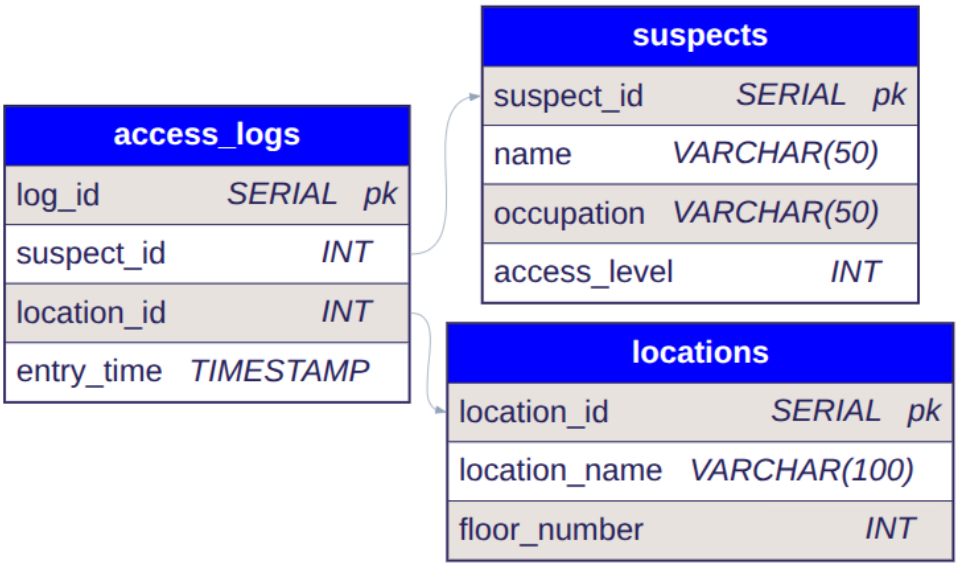
Database Setup Code (For Lecture Demo)

This is just to understand the structure. The sql for your task is different.

```
CREATE TABLE suspects (
    suspect_id SERIAL PRIMARY KEY,
    name VARCHAR(50),
    occupation VARCHAR(50),
    access_level INT
);

CREATE TABLE locations (
    location_id SERIAL PRIMARY KEY,
    location_name VARCHAR(100),
    floor_number INT
);

CREATE TABLE access_logs (
    log_id SERIAL PRIMARY KEY,
    suspect_id INT REFERENCES suspects(suspect_id),
    location_id INT REFERENCES locations(location_id),
    entry_time TIMESTAMP
);
```



2 Technique 1: Connecting Evidence (SQL Joins)

Detectives use Joins to connect a person (Suspect) to a place (Location) via evidence (Logs).

2.1 INNER JOIN (Confirmed Presence)

The `INNER JOIN` returns records only when there is a match in **both** tables. It answers: "Who definitely swiped their card?"

Query: Show Suspect Names and the Location IDs they visited.

```
SELECT S.name, A.location_id, A.entry_time
FROM suspects S
INNER JOIN access_logs A ON S.suspect_id = A.suspect_id;
```

Result: *Note: Sieam (ID 4) is not listed because he has no log entry.*

name	location_id	entry_time
Zahid	101	14:00
Nafis	102	14:10
Zahid	102	15:00
Tabib	101	09:00

2.2 NATURAL JOIN (The Implicit Connection)

A `NATURAL JOIN` is a shortcut. It automatically performs an `INNER JOIN` by looking for columns with the **same name** and data type in both tables. In our case, both **suspects** and **access_logs** share the column `suspect_id`.

Query: Join Suspects and Logs without specifying the `ON` condition.

```
SELECT name, entry_time
FROM suspects
NATURAL JOIN access_logs;
```

Result: The database automatically detected `suspect_id` as the common key.

name	entry_time
Zahid	14:00
Nafis	14:10
Zahid	15:00
Tabib	09:00

Detective's Warning: Use this technique with caution! If your tables accidentally share a column name that isn't a foreign key (e.g., both tables have a generic `comment` or `created_at` column), the Natural Join will try to match those as well, leading to incorrect results or empty tables.

2.3 Joining Three Tables

To see the **Location Name** instead of the ID, we join a third table.

Query: Show Suspect Name and Location Name.

```
SELECT S.name, L.location_name
FROM suspects S
JOIN access_logs A ON S.suspect_id = A.suspect_id
JOIN locations L ON A.location_id = L.location_id;
```

3 Technique 2: Finding Missing Links

Sometimes, what is **missing** is the most important clue.

3.1 LEFT JOIN

Returns all records from the left table (Suspects), and matched records from the right (Logs). If a suspect has no logs, they appear with NULLs.

Query: List all suspects and their visit times. Include those who did not visit anywhere.

```
SELECT S.name, A.entry_time
FROM suspects S
LEFT JOIN access_logs A ON S.suspect_id = A.suspect_id;
```

Result: *Sieam has NULL entry_time, meaning he left no digital footprint.*

name	entry_time
Zahid	14:00
Zahid	15:00
Nafis	14:10
Tabib	09:00
Sieam	*NULL*

3.2 RIGHT JOIN

Returns all records from the right table. Useful to find logs that don't match a known suspect (e.g., a corrupted ID).

Query: Show all Logs and the associated Suspect Name.

```
SELECT S.name, A.entry_time
FROM suspects S
RIGHT JOIN access_logs A ON S.suspect_id = A.suspect_id;
```

4 Technique 3: Profiling (Aggregates)

4.1 COUNT, SUM, AVG, MAX

Query: Get statistics on the suspects.

```
SELECT
    COUNT(*) AS total_suspects,
    MAX(access_level) AS highest_clearance,
    AVG(access_level) AS avg_clearance
FROM suspects;
```

Result:

total_suspects	highest_clearance	avg_clearance
4	5	2.75

5 Technique 4: Grouping Suspects (GROUP BY)

5.1 GROUP BY with Aggregate

Query: Count how many times **each** suspect swiped their card.

```
SELECT suspect_id, COUNT(*) AS visit_count
FROM access_logs
GROUP BY suspect_id;
```

Result:

suspect_id	visit_count
1 (Zahid)	2
2 (Nafis)	1
3 (Tabib)	1

5.2 GROUP BY with JOIN (Readable Report)

Query: Count visits per Location Name.

```
SELECT L.location_name, COUNT(A.log_id) AS total_visits
FROM locations L
JOIN access_logs A ON L.location_id = A.location_id
GROUP BY L.location_name;
```

Result:

location_name	total_visits
Server Room	2
North Canteen	2

6 Technique 5: Narrowing the List (HAVING)

The `HAVING` clause filters groups **after** the aggregation. It is essential for finding "suspiciously high" activity.

6.1 HAVING Clause

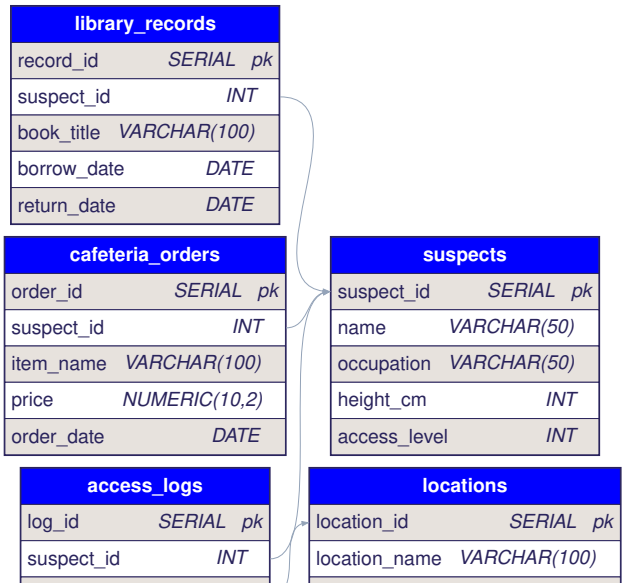
Query: Find suspects who visited locations **more than 1 time**.

```
SELECT suspect_id, COUNT(*) AS visit_count
FROM access_logs
GROUP BY suspect_id
HAVING COUNT(*) > 1;
```

Result: *Only Zahid is listed.*

suspect_id	visit_count
1	2

7 Lab Tasks



1. List all cafeteria orders where the price is greater than 100 Taka. Show the item name and price.

2. Find out what items each person bought. Display **Suspect Name** and **Item Name** by joining `suspects` and `cafeteria_orders`.

3. List all suspect names and their cafeteria orders. Include suspects who have **not ordered** anything (result should show NULL for item).

4. Count how many times each **Location** was visited. Display Location Name and Visit Count.

5. Find the Location Names that have been visited by **more than 1 person**.